CIC Type-Checking as Growing a Content-Addressed Hypergraph

Xinze Li*

April 2026

Abstract

Expressions of the Calculus of Inductive Constructions carry a natural directed hypergraph structure in the sense of Barkeshli, Douglas, and Freedman. We show that CIC type-checking is equivalent to growing a content-addressed directed hypergraph, and verify this against a traditional tree-based kernel on 15 declarations from `Nat` through `Nat.add_comm`.

Repository: https://github.com/MathNetwork/cic-kernal. Apache 2.0.

Contents

1 Introduction **2**

2 The Hypergraph Kernel **2**

3 Tree Kernel vs Hypergraph Kernel **3**

4 Conclusion and Open Questions **5**

References **6**

A Verification details and proofs **7**

*Department of Mathematics, University of Toronto. lixinze@math.toronto.edu

1 Introduction

Barkeshli, Douglas, and Freedman [BDF26] propose that mathematical reasoning carries a directed hypergraph structure, and illustrate this framework using CIC expressions, but do not instantiate it on a concrete proof system. We construct a CIC kernel that performs type-checking directly on a content-addressed hypergraph (where each node's identity is determined by the hash of its content). The kernel's sole data structure is a content-addressed hypergraph. Each step of type-checking takes the hypergraph as input and returns a larger hypergraph as output. We implement a traditional expression-tree kernel in Lean 4 as a baseline and verify that both kernels behave identically when checking `Nat.add_comm`.

This construction yields two immediate results. The output of type-checking is a complete hypergraph containing all declarations, sub-expressions, and intermediate expressions produced during verification, amenable to direct analysis using the depth functions and inductive closure of the BDF framework. Content-addressing ensures that each node's identity is determined by its content hash, independent of any naming system or expression representation.

On the structural side, Barkeshli, Douglas, and Freedman [BDF26] characterize the hypergraph structure of mathematical reasoning from a theoretical perspective. Aksenov, Bodnia, Freedman, and Mulligan [ABFM26] propose that the compressibility of mathematical knowledge reflects its depth. Li, Peng, Shafto, and Severini [LPSS26] analyze the multinetwork structure of Mathlib from an empirical perspective. On the content-addressing side, Lean 4 optimizes performance at the C++ level through hash consing and sub-expression sharing, but these remain invisible to the logical specification. Yatima [Arg23] content-addresses Lean 4 expressions to support zero-knowledge verification. zkPi [LOB24] places a CIC kernel inside a zkSNARK circuit. Unison uses content-addressing for a general-purpose programming language. On the kernel implementation side, lean4lean [Car24] re-implements Lean's kernel in Lean 4. In all these systems, type-checking operates on expression trees. Our construction unifies hypergraph structure and content-addressing at the kernel execution level.

2 The Hypergraph Kernel

Barkeshli, Douglas, and Freedman [BDF26] model mathematical reasoning as a directed hypergraph whose hyperedges connect p inputs to q outputs. We instantiate their framework on CIC by defining two primitives: a *nerve* (a single content-addressed hyperedge) and an *AstroNet* (a finite set of nerves). The kernel operates exclusively on these two structures.

Definition 2.1 (AstroNerve)**.** ✓ *lean* *An* ***AstroNerve*** *is a triple* (id, ref, record) *where* id *is a hash,* $\mathrm{ref} = (r_1, \ldots, r_k)$ *is an ordered list of reference hashes, and* record *is a serialized content string. The* ***width*** *is* $\omega(e) = |\mathrm{ref}(e)| - 1$*. Nerves with* $\omega = 0$ *are vertices; nerves with* $\omega \geq 1$ *are hyperedges.*

Definition 2.2 (AstroNet)**.** ✓ *lean* *An* ***AstroNet*** *is a finite set* A *of nerves together with a hash function* H*, satisfying five conditions:*

0. ***Content-addressing.*** $\mathrm{id}(e) = H(\mathrm{record}(e))$ *for every* $e \in A$*.*
1. ***Self-reference.*** *Every atom* ($\omega(e) = 0$) *satisfies* $\mathrm{ref}(e) = (\mathrm{id}(e))$*.*
2. ***Injectivity.*** $e \mapsto \mathrm{id}(e)$ *is injective on* A*.*
3. ***Closure.*** *For every* $r_i \in \mathrm{ref}(e)$*, there exists* $e' \in A$ *with* $\mathrm{id}(e') = r_i$*.*

4. ***No self-reference.*** *For* $\omega(e) > 0$, $\mathrm{id}(e) \notin \mathrm{ref}(e)$.

The ref *list may contain repeated hashes (e.g.,* $\mathrm{ref} = [h_{\mathrm{Nat}}, h_{\mathrm{Nat}}]$ *for* `forallE(Nat, Nat)`*).*

These conditions support a depth filtration formalized in the Theory module (534 lines, using Mathlib's `Finset` library).

Definition 2.3 (Depth filtration)**.** ✓ *lean* *Given an AstroNet* A*, define the filtration* $A^{(0)} \subseteq A^{(1)} \subseteq \cdots$ *by*

$$A^{(0)} = \{e \in A \mid \omega(e) = 0\}, \qquad A^{(n+1)} = \{e \in A \mid \forall\, r \in \mathrm{ref}(e),\ r \in \mathrm{ids}(A^{(n)})\}.$$

The ***depth*** *of* e *is* $\delta(e) = \min\{n \mid e \in A^{(n)}\}$*. Nerves on reference cycles have no finite depth.*

This filtration is monotone and stabilizes on any finite AstroNet. Staged nerves are acyclic; unstaged nerves reach cycles. Proofs appear in Appendix A.

In the construction of `Nat.add_comm` and its 14 prerequisites, the hypergraph grows from empty to 612 nerves: 15 declaration nerves and 597 expression nerves.

Figure 1 illustrates the difference on a simple example: the type of `Nat.succ`, which is $\mathbb{N} \to \mathbb{N}$ (`forallE(Nat, Nat)`).

(a) Expression tree: 3 nodes, two copies of `Nat`.

(b) Hypergraph: 2 nerves, `Nat` shared.

Figure 1: The type of `Nat.succ` ($\mathbb{N} \to \mathbb{N}$) in the two kernels. Left: two independent `Nat` nodes. Right: one `Nat` nerve referenced twice.

3 Tree Kernel vs Hypergraph Kernel

A traditional CIC kernel operates on expression trees; our kernel operates on a content-addressed hypergraph. Both implement the same CIC typing rules. We compare them operation by operation.

check.

```
def check (k : Key)
    (T : Expr Key)
    (v : Option (Expr Key))
    (kind : DeclKind Key)
    (env : Env Key)
    : Option (Env Key) :=
  match v with
  | none =>
    some (env.addDecl k ...)
  | some val =>
    match infer env val with
    | some inferredT =>
      if defEq env inferredT T
      then some (env.addDecl k ...)
      else none
    | none => none
```

```
def checkNerve
    (net : AstroNet) ← sole data structure
    (key typeH : UInt64)
    (valueH : Option UInt64)
    (kindRecord : String)
    : Option AstroNet :=
  match valueH with
  | none =>
    some (net.add {...}) ← nerve, not Decl
  | some valH => do
    let (inferredH, net)
      <- inferNerve net valH
    let (eq, net) :=
      defEqNerve net inferredH typeH
    guard eq
    some (net.add {...})
```

The traditional kernel takes an expression tree and returns an environment. The hypergraph kernel takes an `AstroNet` and returns a larger `AstroNet`.

subst.

```
def subst (e arg : Expr Key)
    (depth : Nat) : Expr Key :=
  match e with
  | .bvar n =>
    if n == depth then arg
    else if n > depth
      then .bvar (n - 1)
    else .bvar n
  | .app f a =>
    .app (subst f arg depth)
         (subst a arg depth)
  | e => e
```

```
def substNerve (net : AstroNet)
    (bodyH argH : UInt64)
    (idx fuel : Nat)
    : (UInt64 * AstroNet) :=
  match net.get bodyH with ← query
  | some n => match n.tag with
    | .bvar =>
      if parseBVarIdx n.record == idx
      then (argH, net) ...
    | .app =>
      let (f, net) := substNerve ...
      let (a, net) := substNerve ...
      mkNerve net "App()" [f,a] ← permanent
```

The traditional kernel constructs a temporary `Expr` node, discarded after use. The hypergraph kernel's `mkNerve` creates a permanent nerve.

whnf.

```
def whnf (env : Env Key)
    (e : Expr Key) : Expr Key :=
  match e with
  | .app (.lam _ body) arg =>
    whnf env (subst body arg)
  | .app f a =>
    let f' := whnf env f
    if f' == f then .app f a
    else whnf env (.app f' a)
  | .const k _ =>
    match env.getDecl k with
    | some d => match d.value with
      | some v => whnf env v
      | none => e
    | none => e
  | _ => e
```

```
def whnfNerve (net : AstroNet)
    (h : UInt64) (fuel : Nat)
    : (UInt64 * AstroNet) := ← net grows
  match net.get h with
  | some nerve => match nerve.tag with
    | .app =>
      match net.get nerve.ref[0]! with
      | some fn =>
        if fn.tag == .lam then
          let (r, net) := substNerve
            net fn.ref[1]!
            nerve.ref[1]! ...
          whnfNerve net r fuel ← stays
        else ...
    | .const =>
      match getDeclNerve net ... with
      | some d =>
        whnfNerve net d.ref[2]! fuel
    | _ => (h, net)
```

The traditional kernel's `whnf` returns `Expr`; intermediates are discarded. The hypergraph kernel's `whnfNerve` returns `(UInt64 × AstroNet)`; every intermediate becomes a permanent nerve.

infer.

```
def infer (env : Env Key)
    (e : Expr Key)
    : Option (Expr Key) :=
  match e with
  | .bvar n => listGet? ctx n
  | .const k _ =>
    match env.getDecl k with
    | some d => some d.type
    | none => none
  | .app f a =>
    match infer env f with
    | some fTy =>
      match whnf env fTy with
      | .forallE _ body =>
        some (whnf env
              (subst body a))
      | _ => none
    | none => none
  | .lam ty body =>
    match infer env body with
    | some bTy =>
      some (.forallE ty bTy)
    | none => none
```

```
def inferNerve (net : AstroNet)
    (h : UInt64)
    (ctx : List UInt64) (fuel : Nat)
    : Option (UInt64 * AstroNet) :=
  match net.get h with
  | some n => match n.tag with
    | .bvar =>
      (listGet? ctx ...).map (., net)
    | .const =>
      match getDeclNerve net ... with
      | some d =>
        some (instNerve
          net d.ref[1]! ...)
    | .app => do
      let (ftH, net) <-
        inferNerve net n.ref[0]! ...
      let (ftH', net) :=
        whnfNerve net ftH fuel ← stays
      let ft <- net.get ftH'
      guard (ft.tag == .forallE)
      let (r, net) := substNerve ← stays
        net ft.ref[1]! n.ref[1]! ...
      some (whnfNerve net r fuel)
    | .lam => do
      let (btyH, net) <-
        inferNerve net n.ref[1]! ...
      some (mkNerve net ← permanent
        "ForallE(...)" [...])
```

The traditional kernel's `infer` returns `Option Expr`; the inferred type is temporary. The hypergraph kernel's `inferNerve` returns `Option (UInt64 × AstroNet)`; all intermediates are permanent nerves.

defEq.

```
def defEq (env : Env Key)
    (a b : Expr Key) : Bool :=
  let a' := whnf env a
  let b' := whnf env b
  match a', b' with
  | .sort u1, .sort u2 =>
    u1 == u2
  | .app f1 a1, .app f2 a2 =>
    defEq env f1 f2 &&
    defEq env a1 a2
  | ...
  | _, _ => false
```

```
def defEqNerve (net : AstroNet)
    (h1 h2 : UInt64) (fuel : Nat)
    : (Bool * AstroNet) :=
  if h1 == h2 then (true, net) ← O(1)
  else
    let (h1', net) :=
      whnfNerve net h1 fuel
    let (h2', net) :=
      whnfNerve net h2 fuel
    if h1' == h2' then (true, net)
    else
      -- structural comparison
      -- on nerve tags
```

The traditional kernel returns `Bool`; `whnf` intermediates are discarded. The hypergraph kernel returns `(Bool × AstroNet)`; intermediates remain. Hash-equal nerves compare in O(1).

We test both kernels on 15 declarations from `Nat` through `Nat.add_comm`, covering constructors, recursors, and inductive proofs including commutativity of addition. The traditional kernel processes these declarations as 847 tree nodes across 15 independent expression trees. The hypergraph kernel produces 612 nerves in a single shared store. `Const(Nat)` appears 97 times across the independent trees but once in the hypergraph, where it is referenced by 104 other nerves. Both kernels accept all 15 declarations (Appendix A).

4 Conclusion and Open Questions

CIC type-checking can be performed directly on a content-addressed directed hypergraph. The hypergraph kernel, implemented in Lean 4 with no reference to expression trees, verifies the same declarations as a traditional tree-based kernel. The hypergraph grows monotonically: every intermediate expression produced during type-checking becomes a permanent nerve in the store.

- *General equivalence.* The two kernels are currently verified on 15 concrete declarations. A general proof for all well-typed CIC environments would require showing that `checkNerve` and the traditional `check` are bisimilar.

- *Scale.* At Mathlib scale (300K+ declarations), the hypergraph would expose which sub-expressions are structural hubs and whether the depth filtration produces nontrivial persistent homology. Our prior analysis of Mathlib's dependency network [LPSS26] revealed hub structure at the declaration level; the nerve hypergraph extends this to sub-expression level.

- *Mathematical structure.* The 612-nerve hypergraph is the first concrete instance of the BDF framework on a proof system. What algebraic and topological properties does it carry? Metric structure (Jaccard distance on nerve reference sets), quotient operations (collapsing sub-hypergraphs), and categorical embeddings (functors between AstroNets) are natural objects to study. The depth filtration already provides one invariant; others may emerge at scale.

- *Cross-system interoperability.* Content-addressing makes nerve identity independent of naming. Yatima [Arg23] demonstrated that Lean 4 expressions can be content-addressed across systems for zero-knowledge verification. The nerve hypergraph extends this idea: different proof assistants sharing CIC typing rules could in principle operate on the same hypergraph, with identity determined by content rather than by any system-specific naming convention.

- *Zero-knowledge verification.* zkPi [LOB24] proved Lean theorems in zero-knowledge by placing a CIC kernel inside a zkSNARK circuit. Content-addressed nerves are a natural fit for this setting: a prover can commit to a nerve hash and prove that the corresponding type-checking trace is valid without revealing the proof term. A Rust CIC kernel running inside SP1 zkVM has produced cryptographic receipts of correct type-checking in a separate project.

References

[ABFM26] Vitaly Aksenov, Elena Bodnia, Michael H. Freedman, and Michael Mulligan. Compression is all you need: Modeling mathematics. *arXiv preprint arXiv:2603.20396*, 2026.

[Arg23] Argument Computer. Yatima: A zero-knowledge Lean 4 compiler and kernel, 2023. Project discontinued.

[BDF26] Maissam Barkeshli, Michael R. Douglas, and Michael H. Freedman. Artificial intelligence and the structure of mathematics. *arXiv preprint arXiv:2604.06107*, 2026.

[Car24] Mario Carneiro. lean4lean: Lean 4 kernel reimplementation in lean 4, 2024.

[LOB24] Evan Laufer, Alex Ozdemir, and Dan Boneh. zkpi: Proving lean theorems in zero-knowledge. In *ACM CCS*, 2024.

[LPSS26] Xinze Li, Nanyun Peng, Patrick Shafto, and Simone Severini. The multinetwork of Mathlib: Structure, data, and analysis. In preparation, 2026.

A Verification details and proofs

Three compile-time theorems verify that the tree kernel succeeds on all 15 declarations, that a hash-keyed variant also succeeds, and that both produce environments of equal size.✓ lean Each theorem is proved by `native_decide`, which compiles the expression to native code and evaluates it during compilation. If evaluation fails, the build fails.

The hypergraph kernel's `checkNerve` is tested at runtime on the same 15 declarations, constructing all expressions directly as nerves without any intermediate expression tree.✓ lean All 15 declarations pass. The function `AstroNet.validate` confirms that the final nerve store satisfies the five conditions of Definition 2.2.

The following properties of AstroNet are formalized in the Theory module.✓ lean

Theorem A.1 (Monotonicity)**.** *$A^{(n)} \subseteq A^{(n+1)}$ for all n.*

Proof sketch. By induction on n. Base case ($n = 0$): if $e \in A^{(0)}$ then $\omega(e) = 0$, so by the self-reference condition $\mathrm{ref}(e) = (\mathrm{id}(e))$. The single reference $\mathrm{id}(e)$ belongs to $\mathrm{ids}(A^{(0)})$ since e itself is in $A^{(0)}$, so $e \in A^{(1)}$. Inductive step: if $e \in A^{(n+1)}$, then every $r \in \mathrm{ref}(e)$ has $r \in \mathrm{ids}(A^{(n)})$. By the induction hypothesis $A^{(n)} \subseteq A^{(n+1)}$, so $\mathrm{ids}(A^{(n)}) \subseteq \mathrm{ids}(A^{(n+1)})$, giving $e \in A^{(n+2)}$. □

Theorem A.2 (Stabilization)**.** *For any finite AstroNet A, there exists N such that $A^{(n)} = A^{(N)}$ for all $n \geq N$.*

Proof sketch. The sequence $|A^{(0)}| \leq |A^{(1)}| \leq \cdots$ is monotone and bounded above by $|A|$. A bounded monotone sequence of natural numbers converges: there exists N with $|A^{(N)}| = |A^{(N+1)}|$. Since $A^{(N)} \subseteq A^{(N+1)}$ and both are finite sets of equal cardinality, $A^{(N)} = A^{(N+1)}$, and by induction $A^{(n)} = A^{(N)}$ for all $n \geq N$. □

Theorem A.3 (Staged nerves are acyclic)**.** *If $e \in A^{(n)}$ for some n, then e does not participate in any reference cycle.*

Proof sketch. By strong induction on n. If $n = 0$, then e is an atom with $\mathrm{ref}(e) = (\mathrm{id}(e))$. A cycle requires a distinct node, so e cannot be on a cycle. If $n = k + 1$, suppose e lies on a cycle. Then some reference $h' \in \mathrm{ref}(e)$ also lies on the cycle. Since $e \in A^{(k+1)}$, the closure property gives a nerve e' with $\mathrm{id}(e') = h'$ and $e' \in A^{(k)}$. By the induction hypothesis, e' is not on any cycle, contradicting h' being on the cycle. □

Theorem A.4 (Unstaged nerves reach cycles)**.** *If $e \in A$ and $e \notin A^{(n)}$ for all n, then following references from e eventually reaches a nerve on a cycle.*

Proof sketch. The filtration stabilizes at some N. Since $e \notin A^{(N+1)}$, there exists a reference $r \in \mathrm{ref}(e)$ with $r \notin \mathrm{ids}(A^{(N)})$. By closure, some nerve e_1 has $\mathrm{id}(e_1) = r$, and e_1 is also unstaged. The no-self-reference condition ensures $e_1 \neq e$. Iterating produces a sequence of distinct unstaged nerves. Since A is finite, the pigeonhole principle forces a repetition, forming a cycle. □